Model Switching on the Phone

What we are building, explained without jargon

Prepared by Anagha

What is in this document

1 The idea, using gears

A car changes gear for the hill it is on. Same idea.

2 What the app is deciding

Three jobs the app does during a journey.

3 The three levels

Light, Standard and Strong — and what each one is good for.

4 The four things the phone looks at

Signal, battery, time left, and how well we know you.

5 The switching rules

Six plain if-this-then-that rules. This is the heart of it.

6 One journey, start to finish

Fourteen minutes on the metro, minute by minute.

7 Three more moments

A low battery, a delayed train, and a brand-new user.

> **One thing to keep in mind while reading**
>
> Nothing here changes what the commuter sees on screen. An offer still looks like an offer. All that changes is **which piece of software chose it**, and the app makes that choice sensibly instead of always using the same one.

1 The idea, using gears

A car has more than one gear. Low gear for pulling away and climbing hills: lots of power, but noisy and thirsty. High gear for the motorway: quiet and efficient, but useless on a steep hill. Nobody thinks a car should have one gear and stay in it.

A phone running an app is in the same situation, and the “hill” keeps changing during a single journey. The train goes into a tunnel and the internet disappears. The battery drops. The commuter is two minutes from getting off. Each of those calls for something different from the app.

So instead of one model doing every job in every situation, we put a small set of models on the phone and let the app change gear.

	A car	The LearnPerk app
What changes around it	Hills, traffic, motorway	Signal, battery, how much journey is left
What it switches between	Low gear, middle gears, top gear	Light model, Standard model, Strong model
Who decides	The driver, or the automatic gearbox	The app decides by itself, a few times a minute
What the passenger notices	Almost nothing. The car just keeps going.	Almost nothing. The app just keeps working.

The comparison is not perfect, but it holds where it matters: you switch because the situation changed, and the passenger should not have to think about it.

> **Why not just always use the strongest model?**
>
> Because the strongest model needs things that are not always there. Some need the internet, and the metro spends a lot of its time in tunnels. Some need a decent amount of battery. Some need to know quite a lot about the person, which we do not on their first day. When those things are missing, the strong model does not give a slightly worse answer — it gives **no answer at all**, and the commuter sees an empty screen.

2 What the app is deciding

During a journey the app is quietly making three kinds of decision. Each one is made by its own model, and each one can switch independently of the others.

The job	The question it answers	How often
1. Choosing offers	Of all the deals available near this person right now, which one do we show, and in what order? This is the one that makes money, so it is the one we care most about.	A few times per journey
2. Choosing content	Which article or video goes at the top of the feed? Something short if the journey is nearly over, something in a topic they actually read.	Every time the feed refreshes
3. Answering questions	The commuter types *"why is my tram delayed?"* and the app answers. This one is the most expensive to run, so it is the first thing we turn down.	Only when someone asks

Three jobs, three separate sets of models. The app can be running a Strong model for offers and a Light one for content at the same moment — that is fine and often correct.

> **A note on scope**
>
> For the first build we are doing switching on **jobs 1 and 2 only** — offers and content. Job 3, the question-answering, uses much bigger models and is a bigger piece of work. It comes after.

3 The three levels

For each job we keep three models on the phone. They all do the same job. They differ in how much they need and how good they are.

Choosing offers

Strong	**The Smart Picker** Uses everything we know about this person, including how far they will walk out of their way and how close they are to their next reward. The best answers — when it can run.
Standard	**The Everyday Picker** The one that runs most of the time. Knows this person reasonably well, works without the internet, and is quick on any phone. This is the default.
Light	**The Simple Picker** A small lookup: people like you, at this time of day, tend to take offers like this. Barely personal, but it is instant, needs nothing, and always works.

Choosing content

Strong	**The Smart Feed** Reads the actual text of articles to find ones close to what this person likes, even for articles nobody has opened yet.
Standard	**The Everyday Feed** Learns from what they have opened, how long they stayed, and what they skipped. The default when there is internet.
Light	**The Simple Feed** Sorts whatever is already saved on the phone by the topics they chose when they signed up, and by whether it is short enough to finish before they get off.

> **The important one is the Light level**
>
> It is tempting to think of the Light models as the sad fallback. They are the opposite: they are the reason the app never goes blank. They work in a tunnel, on a five-year-old phone, at 9% battery, on someone’s first day. Whatever else fails, one of these can always run, and that is a rule we are building in deliberately.

For the developers reading this, the plain names above map to real things:

Plain name	What it actually is	Rough size
The Simple Picker	A pre-computed lookup table, refreshed overnight	about 60 KB
The Everyday Picker	A small gradient-boosted model, quantised	about 180 KB
The Smart Picker	A larger gradient-boosted model with more features	about 1.4 MB
The Simple Feed	Sort by declared interest and article length	about 4 KB
The Everyday Feed	A gradient-boosted model over reading history	about 220 KB
The Smart Feed	Sentence embeddings plus nearest-neighbour search	about 23 MB

Sizes are rough targets to give a sense of scale, not measured figures. Note the whole offer ladder together is under 2 MB — smaller than a single photo.

4 The four things the phone looks at

To switch sensibly the app has to notice what is going on. It checks four things, every few seconds, and all four are things the phone already knows — we are not asking the commuter anything or collecting anything new.

	What it is	What it tells us
1. Signal	Is there internet right now? Full, patchy, or none at all.	This is the big one on the metro. Underground there is often no signal, so anything that needs the internet simply cannot run.
2. Battery	How much charge is left, and how fast it is draining.	Below roughly a quarter charge, people start closing apps to save their phone. We would rather step down and stay useful than run hot and get shut off.
3. Time left	Roughly how many minutes until they get off this train.	A typical leg is about twelve minutes. With two minutes to go there is no point loading a long article or spending time picking the perfect offer.
4. How well we know them	Are they a regular with months of history, or did they sign up on Tuesday?	The Strong models need history to be any good. For a brand-new user there is nothing to work from, so a simpler model is genuinely the better answer.

Four checks. The app also keeps an eye on how hard the phone is working and how old the phone is, but those mostly just rule out the Strong level on older devices.

> **Nothing new is being collected**
>
> All four of these already exist. Signal strength, battery level and journey time are things the phone and the GVB feed report anyway. “How well we know them” is just a count of things that already happened. The only genuinely new record is a small note saying *which model was used, and why we switched* — and that is information about our software, not about the person.

5 The switching rules

This is the heart of the document. Six rules, in plain if-this-then-that form. They are checked in the order below, and the first one that applies wins.

#	If this is true…	…then use	The plain reason
1	**No signal.** The phone has lost the internet — usually a tunnel.	**Light** for offers. **Light** for content.	Anything that reaches out to the internet cannot work. The Light models were built to run entirely on the phone, so they carry on as normal.
2	**Battery is low.** Under about a quarter, or draining unusually fast.	**Light** for offers. **Light** for content. Stop loading content in advance.	People close apps that eat their battery. Stepping down keeps the app alive for the rest of the journey instead of being shut off halfway.
3	**Almost there.** Less than about two minutes of journey left.	**Light** for offers. **Content switched off.**	Nobody starts a five-minute article with two minutes to go. Offers still make sense if they can be used quickly, so those keep running.
4	**We hardly know them.** New account, or almost no history yet.	**Light** for offers. **Light** for content.	The stronger models work by spotting patterns in what someone has done before. With nothing to look at they do not give a careful answer, they give a random one. The simple one is honestly better here.
5	**Old or slow phone.** Lower-end device.	**Never Strong.** Light or Standard only.	The Strong models run badly on older hardware — slow enough that the commuter would notice. Roughly three in ten phones are in this group, so this is not an edge case.
6	**Everything is fine, and there is time.** Good signal, healthy battery, decent phone, more than ten minutes left, screen actually on.	**Strong** for offers. **Strong** for content.	This is the moment worth spending on. Often it is a delayed train: the person is stuck, looking at their phone, with time to act on a good offer.

*Six rules, colour-coded by where they land: green steps down to Light, grey rules out the top level, blue steps up to Strong. If none of them applies, the app uses **Standard**, which is the normal state and where it spends most of its time.*

Two things the rules do on purpose

They do not only step down. Rule 6 steps *up*. A switching system that only ever makes things smaller is just a brake, and it would miss the best moments. A delayed train with a charged phone is the most valuable situation LearnPerk gets, and the rules should spend more there, not less.

They do not flip back and forth. If the battery hovers right on the line, we do not want the app switching every few seconds. So once a model is chosen it stays for a short minimum time, and the situation has to change clearly — not marginally — before we switch again. In practice this means a handful of switches per journey, not hundreds.

> **The one rule underneath all the others**
>
> There must **always** be something that can run. Whatever the combination of no signal, low battery, old phone and new user, the Light model is built so that it can never be ruled out. The app is never allowed to end up with nothing to show.

6 One journey, start to finish

Lisa takes the metro from Zuid to Centraal. Fourteen minutes, a normal Tuesday morning, phone at 56%. Here is what the app does, minute by minute. She notices none of it.

Time	What is happening	Offers	Content	Why it switched
0:00	On the platform, waiting. Good signal, plenty of battery, twelve-plus minutes ahead of her.	**Standard**	**Standard**	Nothing unusual, so the app sits in its normal state.
3:10	The train goes into the tunnel. **Signal drops to nothing.**	**Light**	**Light**	**Rule 1.** No internet. Both jobs drop to the models that run entirely on the phone.
3:10	Same moment, from Lisa’s side: the feed keeps working and offers keep appearing.	—	—	This is the point. Today she would see an empty feed for the whole tunnel. With switching, she does not.
10:40	Out of the tunnel. **Signal comes back.**	**Standard**	**Standard**	Conditions are normal again, so the app returns to its normal state. It waits a few seconds first to be sure the signal is really back.
12:30	Approaching Centraal. Under two minutes left.	**Light**	**Off**	**Rule 3.** No time to read anything, so content stops. A quick offer she could use at the station still makes sense.
14:00	She gets off.	—	—	Journey over. Everything resets.

Three switching moments across a fourteen-minute journey. The commuter sees a working app throughout, which is the only thing she should notice.

> **What to take from this one**
>
> The interesting moment is 3:10. Today, going into that tunnel means the content feed goes blank until she comes out — roughly seven minutes of a fourteen-minute trip. The switching does not make the feed *better* there, it makes it *exist*. That is the clearest single thing this project does.

7 Three more moments

The same six rules, in three other situations, to show they pull in different directions.

John, 18% battery, on his way home

The situation	Evening metro. Older phone, 18% charge and dropping quickly. Fourteen minutes to go.
What switches	Offers step down to **Light**. Content steps down to **Light**, and loading articles in advance stops.
Which rule	**Rule 2**, the battery rule. **Rule 5** had already ruled out the Strong level on his phone anyway.
The point	John is close to the level where people start shutting apps down. The app spending less here is what keeps it open for the rest of his journey.

Sanne, stuck on a delayed tram

The situation	Peak hour. The tram is eleven minutes late and packed. She is standing, phone in hand, 74% battery, good phone, about twenty-two minutes still to go.
What switches	Offers step **up** to **Strong**. Content steps **up** to **Strong**.
Which rule	**Rule 6.** Everything it asks for is true at once, which does not happen often.
The point	This is the opposite of the other examples, and it is deliberate. She is stuck, attentive and has time to act. If we are ever going to use the best model, it is now. The app goes back to Standard after a few minutes so this does not run all day.

Daan, who signed up nine days ago

The situation	Bus across town, sixteen minutes, good signal, 63% battery, decent phone. Nothing is under any pressure at all.
What switches	Offers step down to **Light**. Content steps down to **Light**. As he uses the app more, they come back up on their own.
Which rule	**Rule 4.** Not because the phone cannot cope — it can — but because we do not know enough about him yet.
The point	This is the example people find least obvious. Everything is fine and we still step down. A Strong model with nothing to learn from does not hedge; it makes a confident guess that happens to be wrong. For someone nine days in, one badly chosen offer is a good way to lose them.

	Lisa	John	Sanne	Daan
What triggered it	Tunnel	Low battery	Delay, and time to spare	Too new
Which way	Down, then back up	Down	**Up**	Down
Rule	1, then 3	2 and 5	6	4
What it gets us	A feed that is not blank	The app survives the trip	The best answer at the best moment	No confident wrong guess

Four people, four different outcomes, one set of rules. Three step down and one steps up — which is the balance we want.